Implementation of Formation Control of Nonholonomic Mobile Robots Using Graph Theoretical Methods

Zhenyu Jiang

August 2025

Link to the demo video: Demo on YouTube.
Link to the code repository: Code on GitHub.

1 Intro

As the increased popularity of cooperative control of multiple systems, many control laws like virtual structure, leader-following, or graph theory are able to provide promising results on controlling multiple machines/robots at once. However, limitations exist when we come to practical usage. The virtual structure approach is to maintain formation while moving, but it requires the formation to act as a virtual structure (one single rigid body), which limits the physical implementation. The leader following approach is easy to implement since all robots just need to follow the trajectories of the leader robot in preset pace, but the downside is when the leader fails, everything fails. Lastly, the graph theoretical approach is turning the system into a graph, where robots are vertices and links between them are edges. This system is able to guarantee the stability of each robot/agent even if they have different internal set up as long as their dynamic can be unified. However, this law only applies to linear systems. This paper extends on the graph theoretical approach and proposes two approaches to adapt non-linear models. The following is a simple implementation for the two approach mentioned in the paper using pygame.

2 Set up

I define my desired formation as follow:
Robot1: (0,0)
Robot2: (20,100)
Robot3: (100, 50)
$L = I - D^{-1}G$

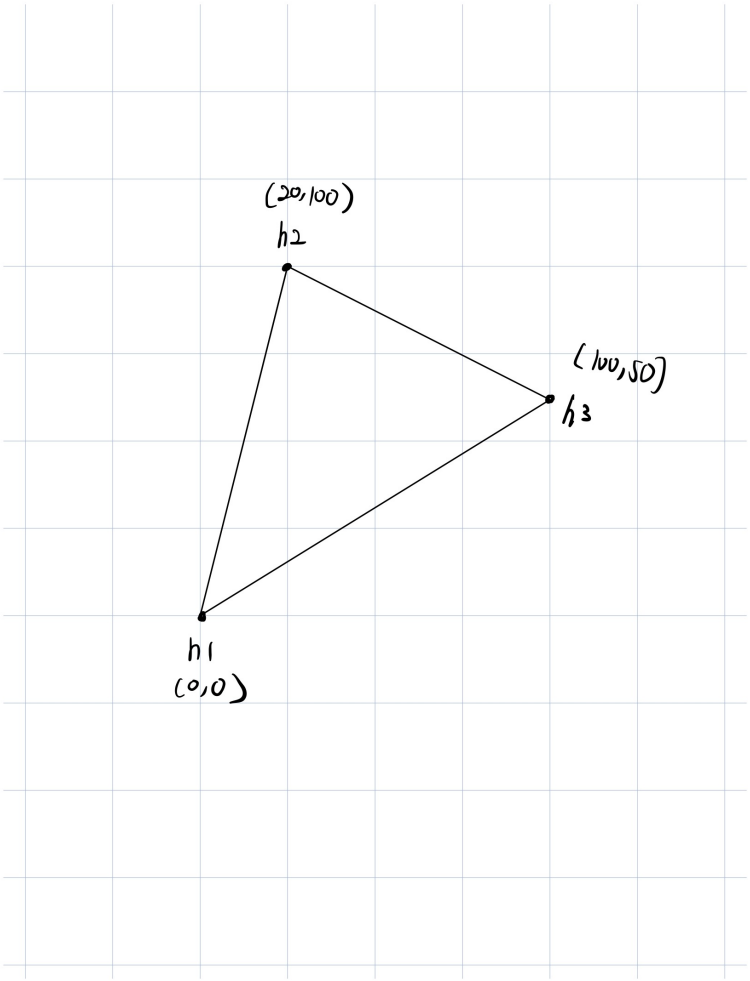


Figure 1: Desired Formation for apporch 1 and 2

$$L = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

The state of each robot is: [x, $\dot{x}$, y, $\dot{y}$]

Then, I apply the simple linear double-integrator as the dynamic for robots, where:

$$\dot{z} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{y} \\ \ddot{y} \end{bmatrix} = A_0 \begin{bmatrix} x \\ \dot{x} \\ y \\ \dot{y} \end{bmatrix} + B_0 \begin{bmatrix} u_x \\ u_y \end{bmatrix}$$

$$A_0 = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad B_0 = \begin{bmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 0 \\ 0 & 1 \end{bmatrix}$$

$$A_v = A_0, B_v = B_0, q_j = [q_{1,j}^T, q_{2,j}^T], r_j = [r_{1,j}, r_{2,j}]$$

$$q_{1,j} = \begin{bmatrix} x \\ dx \end{bmatrix}, q_{2,j} = \begin{bmatrix} y \\ dy \end{bmatrix}$$

I define my desire state $\hbar$ very similar to the paper:

$$\begin{aligned} \hbar_{j,1} &= [h_{j,x}, 0, \ldots]^T \in \mathbb{R}^{n-1} \\ \hbar_{j,2} &= [h_{j,y}, 0, \ldots]^T \in \mathbb{R}^{n-1} \\ \hbar_j &= [\hbar_{j,1}^T, \hbar_{j,2}^T]^T \\ \hbar &= [\hbar_1^T, ...\hbar_m^T]^T, \end{aligned}$$

and

$$\begin{aligned} \rho &= L_1(q - \hbar), \\ L_1 &= L \otimes I_4 \end{aligned}$$

Where:

$$\hbar = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 20 & 0 & 100 & 0 \\ 100 & 0 & 50 & 0 \end{bmatrix}$$

Basically showing the ideal position for the 3 robot should reach in the end

Since my n = 2 (I am not completely sure but I am assuming n in this case is the dimension of the system), so I set my Γ as

$$\Gamma = \begin{bmatrix} g_1 & g_2 \end{bmatrix}$$

$$\Gamma = \begin{bmatrix} [-50.0 & -30.0 & 0 & 0], \\ [0 & 0 & -50.0 & -30.0] \end{bmatrix}^T$$

where g_1 is the gain control feedback for the x direction, and g_2 is for the y-direction. I tried my best to follow the paper set up, but some of the properties, such as dimension of matrix can't work on my current implementation. Therefore, dimensions such as L_1 and Γ might be tweaked a bit to make the implementation work.

3 Approach I

I simply followed the equation as shown on the paper:

$$r = \Gamma L_1 (q - \hbar)$$

However, in order to make it work, our Γ, as paper mentioned, need to be chosen that is Hurwitz stable for each non zero eigenvalue. Otherwise, robots trajectories will be unstable and blew out. Lastly, my set up can't accelerate too hard using this formula

$$\dot{q} = (Aq + Br) * DT$$

If the acceleration is too big, like $DT \geq 0.05$, the formation will be unstable.

Here is the result after applying the control:

4 Results/observation

As shown from figures and demo, robots are able to quickly form the desired formation in seconds. Further, the formation can go to any desired location on the graph. Unfortunately, I was only able to validate the methods with linear model, however, the core idea of this paper, the graph-theoretic formation control design for formation control, is valid (since we can always turn the non-linear system to linear by applying proper linearization method). By implementing the method, I found the following parameters are tricky:

- A_0 (state transition matrix) and B_0 (input matrix) can vary depending on the implementation. As shown in my set up, my B_0 is a matrix of 4 by 2 rather than the matrix of n by 1 in the paper.
- The size of the identity matrix can also vary depending on the implementation. Although in paper it says $A_v = I_2 \otimes A_0$, the dimension does not match my system setup after the computation.

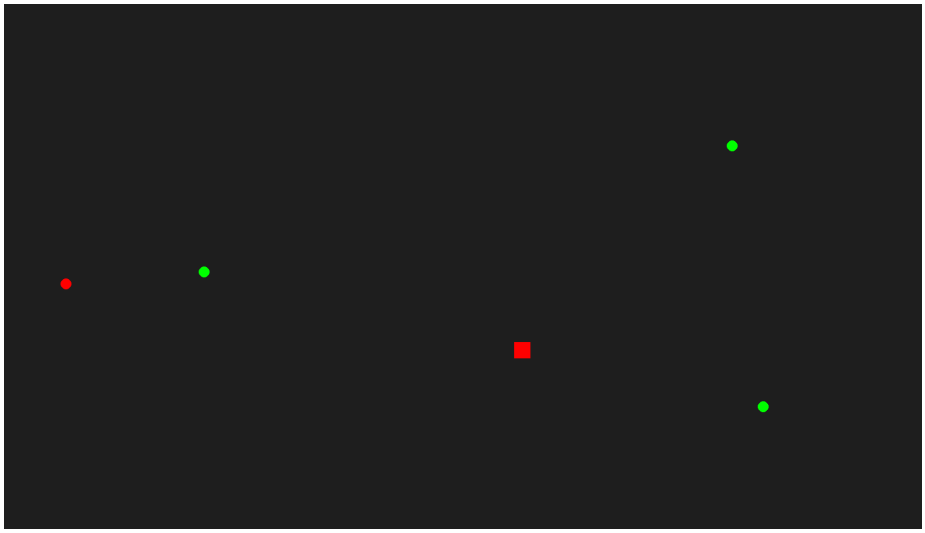


Figure 2: Demo on pygame, 3 robots are randomly generated on the screen and eventually come together and form a formation. Green dots are robots, red dot is the desire trajectories that the formation should follow, and the red rectangle is the target location

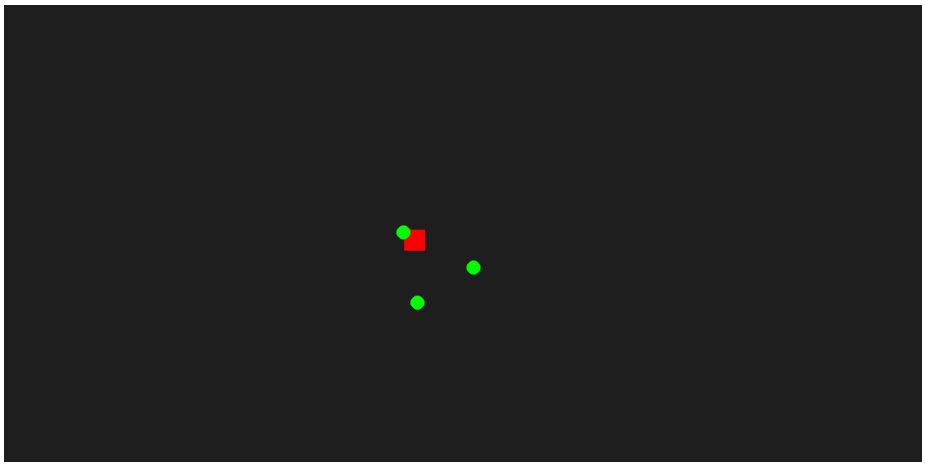


Figure 3: The formation robots reached the target rectangle and stop. The top left robot is assign as "leader", but any robot can be set as the leader.

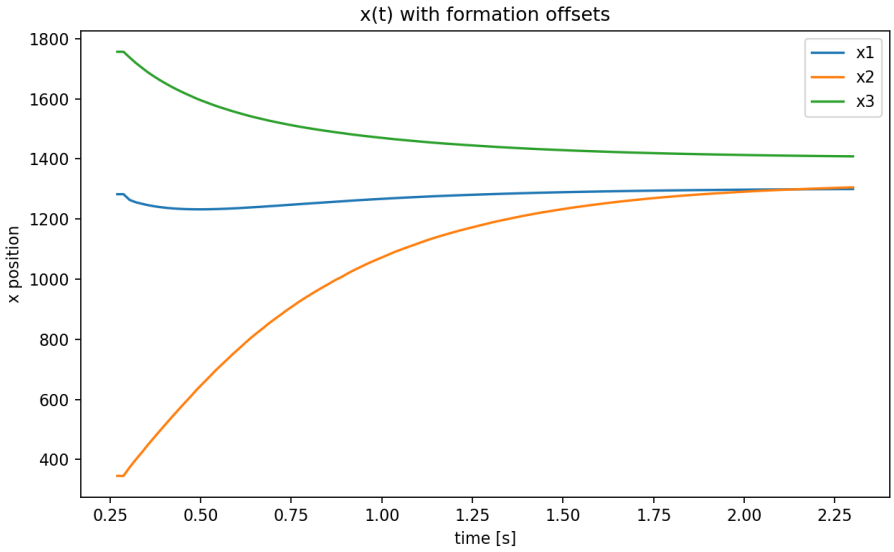


Figure 4: Robots' x positions overtime using controller from Approach I

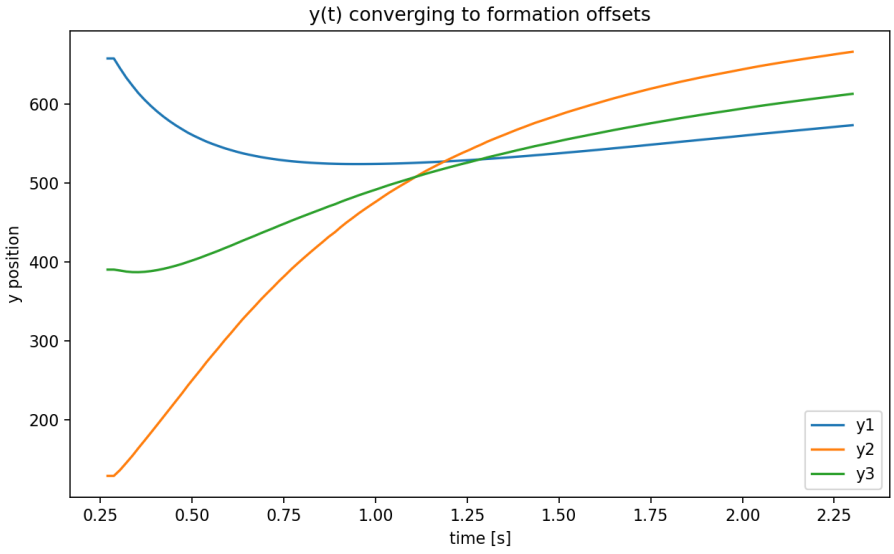


Figure 5: Robots' y positions overtime using controller from Approach I

- Γ need to be chosen that is Hurwitz stable for each non-zero eigenvalue for the system to work, and nothing will work unless this is satisfied because it guarantees the local stability for the system.

5 Approach II

The set up is very similar to Approach I, and the only difference is the following:

$$\Gamma = \begin{bmatrix} g_1 & g_2 \end{bmatrix}$$

$$\Gamma = 0.1 * \begin{bmatrix} [-0.91 & -1.515 & 0 & 0], \\ [0 & 0 & -0.91 & -1.515] \end{bmatrix}^T$$

While my A_0, B_0, and $L_G = L$ remain unchanged. Based on the approach, the control is constructed in 2 steps. The first step is to obtain $u_1 = \dot{\alpha}1 - L_G y_1$, and the second step is to use u_1 to obtain $u_2 = diag[u_{1,1}..., u_{1,m}]\Gamma(\bar{z} - h)$.
Since $\dot{\alpha}$ needs to be continue differentiable, I choose $\dot{\alpha} = 0.5t$. I tried to choose something to prevent my system from being unstable such as $\dot{\alpha} = 0.5 * e^{-t}$ at the very beginning, but I found out that as long as $\dot{\alpha}$ can converge to 0 (meaning the system can be stable) as time goes infinity, the choice of alpha doesn't really matter in my set up. The rest I just follow the paper:

$z_{1,j} = x_j, \bar{z} = [x, dx, y, dy], y_1, j = x_j - 0.5 - h_{j,x}, h_j = [h_{j,x}, 0, h_{j,y}, 0], h = [h_1^T, h_2^T ... h_m^T]$

Noticed that my desire state (h) and $\bar{z}$ set up is a bit different than the paper. The paper only wants the desire state for the y position and $\bar{z}$ only contains dx,y,dy for robots; unfortunately, if I apply that to my system, my robot will end up having the same x. I guess that might be due to the different implementation set up. Then I update my robot states by:

$$\dot{q} = A_v q + B_v u_2, q = [x, dx, y, dy]$$

6 Results/observation

As shown in the figures and demo, **Approach 2** provides a controller that quickly brings the robots into formation. This two-step controller design relies on graph-theoretic properties of the **graph Laplacian**. By construction, each row of a Laplacian matrix sums to zero, which guarantees that the all-ones vector $\mathbf{1}$ lies in its nullspace ($L\mathbf{1} = 0$). In other words, if we plug any vector of identical values into L, the result is the zero vector:

$$\dot{x} = -Lx = -L(c\mathbf{1}) = -c(L\mathbf{1}) = 0.$$



Figure 6: Demo on pygame, 3 robots are randomly generated on the screen from Approach II

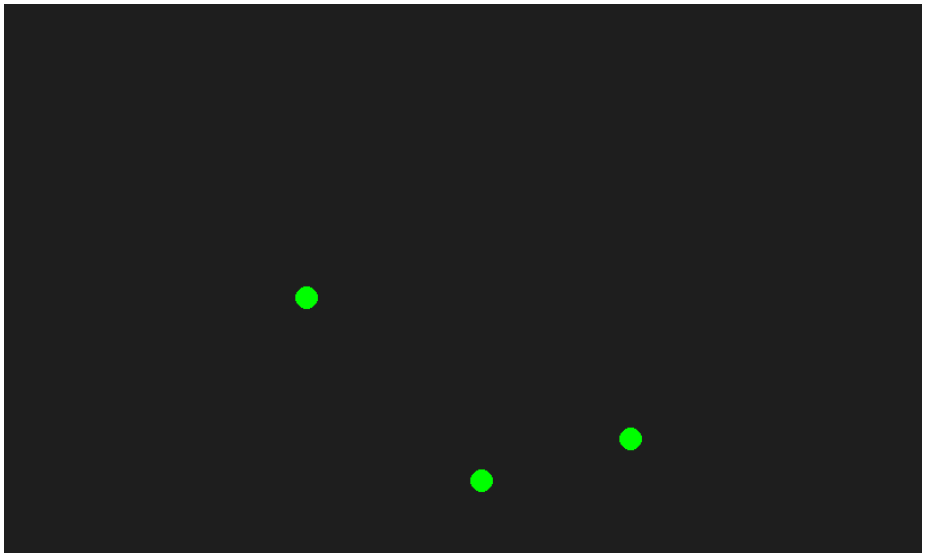


Figure 7: Robots eventually come together and form a formation in just seconds, and then continue to one direction moving as a whole

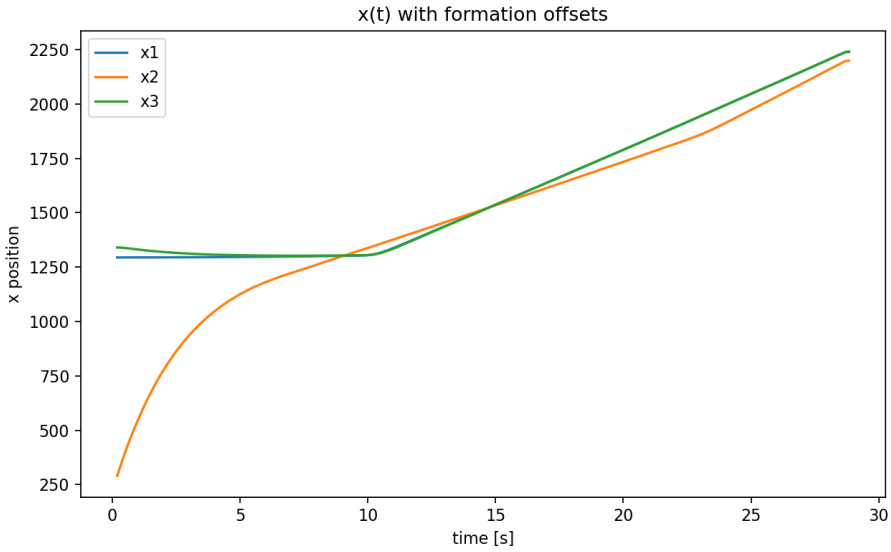


Figure 8: Robots' x positions overtime using controller from Approach II

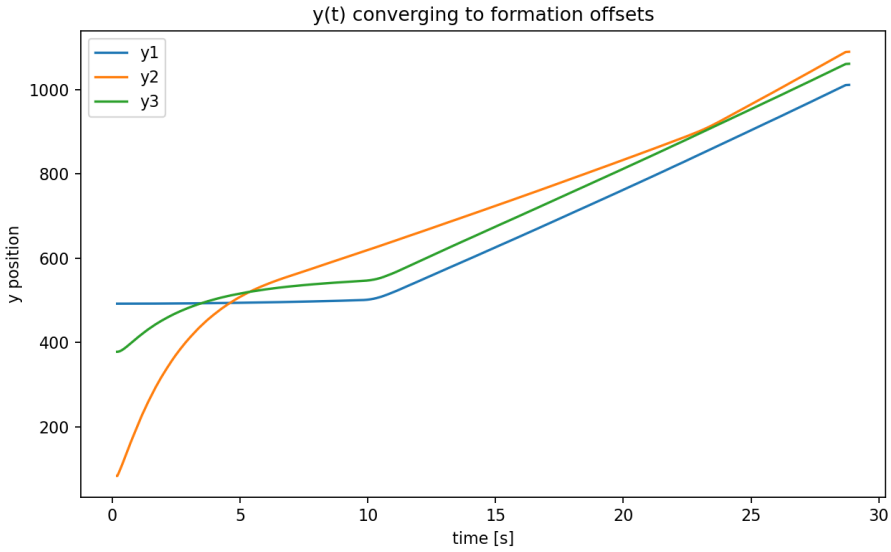


Figure 9: Robots' y positions overtime using controller from Approach II

This means any configuration where all robots share the same x-coordinate (for any constant c) is an **equilibrium**. At equilibrium, every neighbor difference $(x_i - x_j)$ is zero, so the robots have reached their desired relative positions and stop moving. Communication between robots follows the directed graph defined in the setup, and because the graph is **strongly connected and balanced**, the dynamics force any initial condition to approach one of these equilibrium states. The term u_1 implements this consensus: the $-Ly_1$ feedback cancels any motion common to all robots and responds only to differences between neighbors, ensuring that each robot's x coordinate converges to the correct formation value while preserving the specified gaps. Once the x-direction motion is synchronized, we begin regulating each robot's y coordinate using the control input u_2. Again, the Laplacian matrix ignores any motion common to all robots and responds only to the error terms $(z - h)$, feeding the relative disagreements in $\dot{x}$, y, and $\dot{y}$ into the controller. The multiplication by $\text{diag}(u_1)$ ensures that the second-step correction rate scales with the progress of the x-direction consensus, so the y-direction adjustment proceeds in proportion to how quickly the robots agree on their x positions. Even though I knew one of the highlights of the paper is that I can compute u2 simply without the x terms. Unfortunately, I tried my best to implementing only utilizing dx, y, dy, which is the $\bar{z} = [z_{2,j}, ..., z_{n,j}]^T$ part, but my set up couldn't work without the first terms. If I have more time, I will do more attempts and fix this issue. However, there's some issue I encounter while implementing this approch:

- A_0 All the statements I mention as a tricky part for Approach I need to hold in order to implement the second apporoach
- In order to properly show the formation process in the pygame, gamma values and DT need to be chosen in a very small numbers, as shown in my gamma matrix set up. Otherwise, the robots are accelerating too fast to be shown on the screen.
- The most important part is that u_1 needs to be ensured be non-negatives all the time, otherwise the system will lose the Hurwitz property (unstable).

7 Conclusion

My implementation demonstrates that the graphical approach for formation control that is suggested in the paper is valid with linear models. As before, I encountered many challenges such as matrix dimension mismatch, struggling fully following the paper set up, and finding the correct Γ and $\dot{a}$ to stabilize the system. However, I was able to get them to work in a 2D pygame simulation. Further, I also attempted to let the formation go to certain target or rotate while staying as a whole. In this implementation, I was able to get in touch with the Laplacian matrix and utilizing its properties to establish a stable formation control, which is a new world for me in robotic and control. I was able to have

some idea about the multi-agent system and knowing the theory behind the scene, and I wish to go deeper and implementing more complex and impress multi-agents system in the future.

8 Reference

1. Dong, Wenjie, and Yi Guo. "Formation Control of Nonholonomic Mobile Robots Using Graph Theoretical Methods." Cooperative Systems, 20 Mar. 2007, pp. 369–386, https://doi.org/10.1007/978-3-540-48271-022. Accessed 13 Sept. 2025.